DIGITAL EGYPT PIONEERS INITIATIVE

DATA ENGINEERING TRACK · PHASE 5

TechMart Egypt

E-Commerce Data Integration & Analytics Pipeline

Graduation Project Documentation · Phase 1

PROJECT TEAM

Planning · Literature Review · Requirements Gathering

October 2026

Contents

1 Project at a Glance 1
1.1 Project Profile 1
1.2 Team Members 1
2 Project Planning & Management 1
2.1 Idea, Objectives, and Scope 1
2.2 Timeline & Implementation Plan 2
2.3 Risk Management & Mitigation 2
2.4 Key Performance Indicators 2
3 Literature Review 2
3.1 Comparative Analysis 2
3.2 Value Proposition 3
4 Requirements Gathering 3
4.1 Stakeholder Analysis 3
4.2 Functional Requirements 3
4.3 Non-Functional Requirements 3

1 Project at a Glance

TechMart Egypt is a simulated electronics retailer whose fragmented customer, product, and order data will be integrated into a reliable analytics pipeline. The project brings together source ingestion, Python-based data quality, dimensional modeling, incremental processing, and scheduled orchestration in a maintainable batch architecture.

Pipeline objective
Turn heterogeneous files and API data into clean, validated, query-ready sales analytics—delivered through repeatable daily workflows.

1.1 Project Profile

Initiative	Digital Egypt Pioneers Initiative (DEPI)
Track	Data Engineering · Phase 5
Project	TechMart Egypt — E-Commerce Data Integration & Analytics Pipeline
Document	Graduation Project Documentation · Phase 1
Delivery style	Automated batch and incremental data processing

1.2 Team Members

Team member	Email
Sameh Sobhy	samehattia52@gmail.com
Mahmoud Nasr	elghanamhmwd88@gmail.com
Ahmed Abdelghany	ibrahimelyamany013@gmail.com
Mahmoud Samy	Mahmoud_Sami@science.suez.edu.eg
Ahlam Tarek	ahlamtarek468@gmail.com
Mazen Abdelkawy	mazen.abdelkawey@outlook.com

2 Project Planning & Management

2.1 Idea, Objectives, and Scope

Project idea. The team is developing an automated, end-to-end data engineering pipeline for TechMart Egypt. The system consolidates customer profiles, product catalogs, and transactional order logs from files and APIs; cleans and validates records with Python and Pandas; structures the results in a dimensional star schema; and automates daily execution using Apache Airflow.

Core objectives

1. Reliably extract transactional and catalog data from heterogeneous file and API sources.
2. Implement robust data cleaning, type casting, and validation routines with Python and Pandas.

3. Design a scalable analytics data mart comprising Fact Sales, Dim Customer, Dim Product, and Dim Date.
4. Improve performance with incremental loading and database indexing.
5. Automate pipeline execution and monitor task states in Apache Airflow.

Scope. The pipeline supports batch and incremental loads for technology-store sales orders, customer demographics, and product inventories. Real-time stream-processing frameworks are intentionally excluded so the project can focus on core enterprise batch-engineering practices.

2.2 Timeline & Implementation Plan

Schedule	Planned work
Week 1 · Current	Requirements gathering, team-role alignment, repository initialization, and Phase 1 documentation.
Week 2	Core ingestion modules (API fetchers and file parsers) and staging-layer setup.
Week 3	Pandas transformations, data-quality assertions, and relational star-schema creation.
Week 4	Airflow DAG integration, task-dependency scheduling, and error handling.
Week 5	System integration testing, repository cleanup, documentation finalization, and presentation preparation.

2.3 Risk Management & Mitigation

Risk	Mitigation
API rate limits and timeouts	Custom retry logic with exponential backoff and local caching of raw data.
Schema inconsistency and bad data	Pre-load assertions isolate malformed rows and log them in an error-quarantine table rather than failing the entire pipeline.
Orchestration failures	Explicit task sequencing, retries, and failure alerts configured in Airflow.

2.4 Key Performance Indicators

- **Data integrity:** Remove duplicate orders and invalid primary or foreign keys from curated data.
- **Pipeline performance:** Use indexing and incremental loading to reduce data-load and analytical-query times.

3 Literature Review

3.1 Comparative Analysis

Manual cron scripts vs. the proposed approach. Traditional shell scripts provide limited dependency tracking, state management, and operational visibility. Apache Airflow adds explicit task dependencies, retry handling, dynamic scheduling, and monitoring of workflow status.

SaaS ETL tools vs. a custom pipeline. Managed integration services can be quick to set up, but may add recurring licensing costs and vendor lock-in. A custom Python and Airflow solution gives the team direct control over business logic and validation rules, while supporting cost-conscious scaling as project needs evolve.

3.2 Value Proposition

The project bridges inconsistent raw e-commerce inputs and a structured dimensional warehouse. Incremental loading and indexing complement data-quality controls, helping analysts work with clean, high-integrity data that is ready for business-intelligence reporting.

4 Requirements Gathering

4.1 Stakeholder Analysis

Stakeholder	Needs
Data analysts and BI engineers	Clean, structured, query-optimized dimensional tables for dashboards without data anomalies.
Business operations managers	Timely and accurate daily sales metrics, customer purchasing patterns, and inventory status.
Data engineering team	Modular code, descriptive execution logs, and robust pipeline observability.

4.2 Functional Requirements

1. **Ingestion layer:** Connect to external REST API endpoints and parse local CSV and JSON files containing TechMart Egypt transaction data.
2. **Data cleaning:** Validate types, inspect required-value constraints, remove duplicates, and route error records to an isolation table.
3. **Dimensional modeling:** Load transformed data into a star schema containing `Fact_Sales`, `Dim_Customer`, `Dim_Product`, and `Dim_Date`.
4. **Incremental loading:** Process records inserted or updated since the previous successful execution window to optimize resource consumption.
5. **Orchestration:** Use Apache Airflow to sequence and automate extract, transform, load, and validation operations.

4.3 Non-Functional Requirements

- **Performance and optimization:** Daily batch pipelines should use indexes on frequently searched columns, such as `CustomerID` and `OrderDate`, to improve query speed.
- **Maintainability:** Source code should follow clean, modular Python conventions (PEP 8), be version-controlled with Git, and include comprehensive technical documentation.